

ARTIFICIAL INTELLIGENCE

IntelliBot — Context-Aware RAG Chatbot

Build an intelligent chatbot powered by LLMs and Retrieval-Augmented Generation (RAG). The bot answers queries exclusively from a custom knowledge base of uploaded company documents — no hallucinations.

■ Domain Overview

Domain: Artificial Intelligence focuses on building systems that reason, learn, and process natural language — bridging the gap between raw unstructured data and intelligent, human-like digital interaction.

Objective: Create a RAG pipeline that ingests PDF/TXT documents, chunks and embeds them into a vector store, and uses an LLM to generate contextually accurate, grounded answers through a conversational UI.

Expected Outcome: A functional chatbot where users ask domain-specific questions and receive accurate answers sourced exclusively from the uploaded knowledge base, with conversation memory for follow-up questions.

■ ■ Tech Stack

Category	Technology
Language	Python 3.11+
LLM Framework	LangChain or LlamaIndex
LLM Provider	OpenAI API / HuggingFace Inference
Embeddings	OpenAI text-embedding or SentenceTransformers
Vector Database	ChromaDB or FAISS
UI Framework	Streamlit or Gradio
Document Parsing	PyMuPDF (fitz) for PDFs
Chat Memory	LangChain ConversationBufferMemory
Env Config	python-dotenv
Evaluation	RAGAS evaluation framework

■ ■ Architecture & Data Flow

- Ingestion Pipeline: Document loaders > Text splitter > Embedding model > Vector store
- Retrieval Engine: Similarity search fetches top-K relevant chunks per user query
- Prompt Engine: System prompt + retrieved context + chat history > LLM API call
- Chat Memory: ConversationBufferMemory maintains multi-turn context per session
- UI Layer: Streamlit chat interface with file uploader and session management
- Evaluation: RAGAS metrics — faithfulness, answer relevancy, context precision

■ 7-Day Milestone Plan

Day	Goals & Tasks
Day 1	Environment setup, API key config, document loader for PDF/TXT files
Day 2	Text chunking (RecursiveCharacterTextSplitter) and embedding generation pipeline
Day 3	ChromaDB setup — persist embeddings, implement similarity search with cosine scoring
Day 4	RAG pipeline — connect retrieved chunks to LLM prompt for grounded responses
Day 5	Conversation memory — ConversationBufferMemory for contextual follow-up questions
Day 6	Streamlit UI — chat interface, file uploader, session history display, clear button
Day 7	RAGAS evaluation, hallucination testing, UI polish, and final project documentation

■ Deliverables

1. Complete Python source code with clean project structure
2. requirements.txt listing all dependencies
3. Sample knowledge base PDFs (3–5 files) for demo
4. Streamlit app running locally or deployed to Streamlit Cloud
5. RAGAS evaluation score report (faithfulness and relevancy)
6. Video demonstration of chatbot answering domain-specific queries
7. README with architecture diagram and setup instructions

■ Evaluation Rubric

Criteria	Weight	Success Indicators
RAG Pipeline Accuracy	30%	Responses grounded in documents, no hallucinations on known topics
Chat Memory & Context	15%	Follow-up questions handled correctly across conversation turns
UI/UX Design	15%	Clean Streamlit UI, file upload works, chat history displayed well
Feature Completeness	20%	Ingestion, retrieval, generation, and memory all functional
Evaluation & Testing	10%	RAGAS scores reported, edge cases tested, error handling

Presentation & Demo	10%	Live demo, explains RAG architecture clearly, Q&A;
TOTAL	100%	

■ Feature List & Wireframe Notes

- PDF and TXT document ingestion
- Recursive text chunking pipeline
- Vector embedding with SentenceTransformers
- ChromaDB persistent vector store
- Top-K cosine similarity retrieval
- Contextual LLM prompting with RAG
- Multi-turn conversation memory
- Streamlit chat UI with file uploader
- RAGAS evaluation scoring
- Hallucination detection and fallback

■ Resources & References

- LangChain: <https://python.langchain.com/docs/>
- LlamaIndex: <https://docs.llamaindex.ai/>
- ChromaDB: <https://docs.trychroma.com/>
- Streamlit: <https://docs.streamlit.io/>
- RAGAS: <https://docs.ragas.io/>

■ Constraints & Guidelines:

- Bot must only answer from provided documents — no internet knowledge leakage
- Embedding and vector search must complete in under 2 seconds per query
- Must handle PDF parsing errors gracefully without crashing
- API keys must be in .env files — never hardcoded in source code